

CUDA N-Body Simulation

A GPU-accelerated N-body gravity simulation implemented in CUDA, C++, and OpenGL.

This project demonstrates how to parallelize gravitational force calculations across many bodies, integrate their motion over time, visualize the result interactively, and experiment with CUDA performance optimizations.

The current implementation uses direct force evaluation, where every body interacts gravitationally with every other body. This makes the project a useful teaching and benchmarking example for CUDA kernels, shared memory, memory traffic, numerical integration, GPU diagnostics, and CUDA/OpenGL interoperability.

Project Goals

This project is intended to:

- Model gravitational attraction between many bodies
- Offload the expensive force calculation to the GPU
- Provide a readable CUDA reference implementation
- Demonstrate shared-memory optimization
- Demonstrate CUDA/OpenGL interoperability
- Compare CUDA launch configurations and block sizes
- Track numerical stability through physical diagnostics
- Serve as a starting point for larger physics simulations
- Provide a benchmark for future CUDA optimization experiments

Features

- Real SI gravitational constant
- Direct $O(N^2)$ gravitational force calculation
- Leapfrog integration
- Shared-memory `float4` force tiles
- CUDA/OpenGL VBO interoperability
- GPU-computed physical diagnostics
- Energy conservation monitoring
- Momentum monitoring
- Center-of-mass monitoring
- Interactive OpenGL visualization
- Pause and resume
- Single-step forward
- Rewind through previously calculated states
- Mouse camera rotation
- Mouse-wheel zoom
- CUDA event timing
- Configurable CUDA threads per block

Current Simulation Parameters

The current defaults are defined in `src/main.cu`.

Parameter	Value
Bodies	16,384
Steps	2,000
Timestep	100 s
Default threads/block	256
Initial radius	1.0×10^8 m
Initial velocity	0 m/s
Body mass range	0.5×10^{20} to 1.5×10^{20} kg
Softening length	1.0×10^5 m
Gravitational constant	6.67430×10^{-11} m ³ kg ⁻¹ s ⁻²

The full 2,000-step simulation represents:

```
2000 × 100 s = 200,000 s
```

or approximately:

```
55.56 simulated hours
```

Simulation Model

Each body stores:

- Position
- Velocity
- Mass

The body structure is:

```
struct Body {
    float3 pos;
    float3 vel;
    float mass;
};
```

The simulation uses Newtonian gravity.

For body `i`, the acceleration due to body `j` is based on:

$$a_i = G \sum_j m_j (r_j - r_i) / (|r_j - r_i|^2 + \epsilon^2)^{3/2}$$

where:

- G is the gravitational constant
- m_j is the mass of body j
- r_i is the position of body i
- r_j is the position of body j
- ϵ is the gravitational softening length

Softening prevents extremely large accelerations when two simulated bodies become very close together.

Leapfrog Integration

The current version uses leapfrog integration rather than the original Euler update.

Each timestep performs:

1. Half velocity kick
2. Full position drift
3. Recompute acceleration
4. Final half velocity kick

Conceptually:

```
v(t + dt/2) = v(t) + a(t) dt/2

r(t + dt) = r(t) + v(t + dt/2) dt

a(t + dt) = gravity(r(t + dt))

v(t + dt) = v(t + dt/2) + a(t + dt) dt/2
```

Leapfrog is useful for orbital and gravitational simulations because it generally has better long-term energy behavior than a simple explicit Euler integrator.

Repository Structure

```
cuda-nbody/
|
|-- CMakeLists.txt
|-- README.md
|
|-- src/
|   |-- main.cu
|   |-- nbody.cu
|   |-- nbody.cuh
```

```
| | simulation.cu  
| | simulation.h  
| | shader.cpp  
| | shader.h  
|  
└─ build/
```

CMakeLists.txt

Configures the C++, CUDA, GLFW, GLAD, and OpenGL build.

src/nbody.cuh

Contains:

- Physical constants
- **Body** structure
- CUDA kernel declarations

src/nbody.cu

Contains the CUDA kernels for:

- Gravitational acceleration
- Leapfrog kick/drift
- Final velocity kick
- Render-position generation
- Physical diagnostics

src/simulation.cu

Contains:

- Initial body generation
- CUDA error handling
- Diagnostic setup and retrieval

src/simulation.h

Contains:

- **Diagnostics** structure
- Simulation helper declarations

src/shader.cpp

Contains OpenGL shader compilation and shader program setup.

src/shader.h

Contains the shader program structure and shader helper declarations.

[src/main.cu](#)

Contains:

- Program setup
- CUDA allocation
- OpenGL initialization
- CUDA/OpenGL interoperability
- Simulation loop
- Playback controls
- History
- Rendering
- CUDA event timing

Requirements

To build and run this project you need:

- Windows 10 or newer
- NVIDIA GPU with CUDA support
- CUDA Toolkit
- CMake 3.18 or newer
- Visual Studio 2022 Build Tools or compatible MSVC compiler
- vcpkg
- GLFW
- GLAD
- OpenGL

The current development environment uses CUDA 13.3 and Visual Studio 2022 Build Tools.

vcpkg

The project uses the existing vcpkg tree at:

```
C:\Users\Maddie\Documents\GitHub\vcpkg\
```

Install the required libraries once:

```
& "C:\Users\Maddie\Documents\GitHub\vcpkg\vcpkg.exe" install glfw3:x64-windows  
glad:x64-windows
```

Configure

From:

```
C:\Users\Maddie\Documents\GitHub\cuda-nbody\build
```

run:

```
Remove-Item CMakeCache.txt -ErrorAction SilentlyContinue; Remove-Item CMakeFiles -  
Recurse -Force -ErrorAction SilentlyContinue; cmake .. -G "Visual Studio 17 2022"  
-A x64 -  
DCMAKE_TOOLCHAIN_FILE="C:/Users/Maddie/Documents/GitHub/vcpkg/scripts/buildsystems  
/vcpkg.cmake"
```

A successful configuration should end with:

```
-- Configuring done  
-- Generating done  
-- Build files have been written to: ...
```

Build

From the `build` directory:

```
cmake --build . --config Release
```

The Release executable is generated at:

```
build\Release\nbody.exe
```

Run

Run with the default 256 threads per block:

```
.\Release\nbody.exe
```

Or specify the CUDA threads per block:

```
.\Release\nbody.exe 256
```

Examples:

```
.\Release\nbody.exe 32
.\Release\nbody.exe 64
.\Release\nbody.exe 128
.\Release\nbody.exe 256
.\Release\nbody.exe 512
```

The argument controls the number of CUDA threads launched per block.

Controls

Control	Action
Space	Play / pause
Right Arrow	Advance one step
Left Arrow	Rewind one step
Left Mouse + Drag	Rotate camera
Mouse Wheel	Zoom
ESC	Exit

The program starts paused at simulation step 0.

CUDA Execution Model

The gravitational kernel assigns one CUDA thread to each target body.

Each thread computes the total acceleration on its body by summing contributions from all other bodies.

Without optimization, every thread would repeatedly read source bodies from global GPU memory.

The current implementation reduces those reads through shared-memory tiling.

Shared-Memory Force Tiling

Bodies are loaded into shared memory in tiles.

The force kernel uses:

```
extern __shared__ float4 sharedBodies[];
```

Each shared-memory entry stores:

```
x
y
```

```
z
mass
```

Velocity is not loaded into shared memory because velocity is not required for gravitational force evaluation.

That means each force tile uses:

```
16 bytes per source body
```

instead of storing the entire **Body** structure.

For 256 threads:

```
256 × 16 bytes = 4096 bytes
```

of shared memory are used per block for the force tile.

Each CUDA block repeatedly:

1. Loads a tile of source bodies from global memory
2. Synchronizes
3. Reuses the tile for every thread in the block
4. Synchronizes again
5. Loads the next tile

This reduces redundant global-memory traffic.

CUDA/OpenGL Interoperability

The OpenGL vertex buffer is registered with CUDA using:

```
cudaGraphicsGLRegisterBuffer(...)
```

When a new simulation state is rendered:

```
CUDA Body Array
  |
  v
writeRenderPositions kernel
  |
  v
CUDA-mapped OpenGL VBO
  |
  v
OpenGL
```

```

      |
      v
glDrawArrays(GL_POINTS)

```

This allows CUDA to write rendering positions directly into the OpenGL vertex buffer.

It avoids the older path:

```

GPU
|
v
CPU
|
v
OpenGL upload
|
v
GPU

```

for newly calculated frames.

Rewind History

The simulation maintains CPU-side snapshots of previous body states.

This allows:

- Pausing the simulation
- Moving backward through calculated states
- Moving forward through already calculated states

When viewing historical states, the stored CPU state is uploaded to the OpenGL VBO.

The CUDA simulation itself remains at the newest calculated timestep.

With 16,384 bodies and 2,000 stored steps, rewind history requires approximately 0.9-1.0 GB of CPU RAM depending on structure alignment.

Physical Diagnostics

The simulation computes several quantities on the GPU.

Kinetic Energy

$$KE = \sum \frac{1}{2} m v^2$$

Potential Energy

Each body pair is counted once:

$$PE = -G \sum_i \sum_{j>i} m_i m_j / r_{ij}$$

with the same gravitational softening used by the force calculation.

Total Energy

$$E = KE + PE$$

The code compares the current total energy to the initial total energy and reports the percentage drift.

Momentum

The simulation computes:

$$P = \sum m v$$

and reports:

$$|P|$$

For an isolated system initialized with zero total velocity, total momentum should ideally remain close to zero.

Center of Mass

The simulation calculates:

$$R_{COM} = \sum m r / \sum m$$

and reports both the current center of mass and its drift relative to the initial state.

Diagnostic Output

The program periodically prints output similar to:

```
Physics diagnostics
Step:          100
Simulation time: 2.778 hours
Kinetic energy: ...
Potential energy: ...
```

```
Total energy:      ...  
Energy drift:      ...  
|Momentum|:       ...  
Center of mass:    ...  
COM drift:         ...
```

Diagnostics currently run every 10 steps.

Because potential-energy calculation is itself $O(N^2)$, diagnostic frequency can affect overall runtime.

Computational Complexity

This is a direct N-body implementation.

For N bodies, each timestep requires approximately:

$O(N^2)$

force interactions.

For 16,384 bodies:

$16,384^2 = 268,435,456$

possible body interactions per timestep.

Across 2,000 steps:

$268,435,456 \times 2,000$
 $= 536,870,912,000$

or roughly:

537 billion interactions

This is approximately eight times the total force-evaluation workload of the earlier:

8,192 bodies $\times$ 1,000 steps

configuration.

Performance Notes

Direct N-body simulation has quadratic runtime growth:

```
Bodies doubled
  |
  v
Interactions per step  $\approx 4\times$ 
```

If both body count and step count are doubled:

```
2× bodies
×
2× steps
=
approximately 8× total force work
```

This makes the program useful for studying:

- CUDA thread-block sizing
- Memory bandwidth
- Shared-memory reuse
- Occupancy
- Kernel execution time
- Numerical stability
- GPU scaling

Block Size Benchmark

Earlier benchmark results with 8,192 bodies and 1,000 steps:

Threads / Block	Blocks	GPU Time
32	256	503.284 ms
64	128	478.162 ms
128	64	479.112 ms
256	32	460.247 ms
512	16	552.347 ms

The best tested configuration in that benchmark was:

```
256 threads/block
```

with:

460.247 ms

GPU execution time.

These values are retained as historical benchmark data and should not be interpreted as the expected timing for the current 16,384-body configuration.

Shared-Memory Improvement

Earlier measurements comparing the original global-memory implementation with shared-memory tiling:

Kernel	Threads / Block	GPU Time
Original global-memory kernel	256	521.351 ms
Shared-memory tiled kernel	256	466.44 ms average
Shared-memory tiled kernel	256	460.247 ms best

At the same 256-thread configuration, shared-memory tiling reduced the measured runtime by approximately:

10.5%

relative to the original implementation.

The current implementation further reduces shared-memory traffic by storing source bodies as `float4` values containing only position and mass.

CUDA Timing

CUDA events are used to time the physics kernels:

```
cudaEvent_t kernelStart;
cudaEvent_t kernelStop;
```

The measured physics step includes:

- First leapfrog kick/drift
- Gravitational acceleration kernel
- Final leapfrog kick

The program reports:

Total leapfrog kernel time

Average physics step time

This timing is more useful for CUDA optimization than measuring the full interactive application wall time, because OpenGL rendering and VSync can dominate the visible playback time.

Important Implementation Details

- GPU memory is allocated with `cudaMalloc`
- Initial body data is copied with `cudaMemcpy`
- One CUDA thread handles one target body
- Shared-memory tiles reduce repeated global-memory reads
- Gravitational acceleration is calculated entirely on the GPU
- Leapfrog integration is performed with CUDA kernels
- CUDA events measure physics-kernel execution time
- OpenGL rendering uses `GL_POINTS`
- CUDA writes directly into the OpenGL VBO for new states
- CPU snapshots are retained for rewind
- `cudaGetLastError()` is used to validate CUDA kernel launches
- CUDA errors are reported through the `checkCuda()` helper

Performance Tuning

Useful parameters to experiment with include:

Threads Per Block

Try:

```
32
64
128
256
512
```

Performance depends on:

- GPU architecture
- Register use
- Shared-memory use
- Occupancy
- Memory latency
- Warp scheduling

CUDA warps contain 32 threads, so block sizes that are multiples of 32 generally avoid partially filled warps.

That does not mean 32 threads per block is necessarily fastest. Larger blocks can improve latency hiding and shared-memory reuse.

Numerical Stability

The current timestep is:

```
100 seconds
```

Leapfrog improves long-term stability, but timestep selection still matters.

If energy drift becomes too large, useful comparisons include:

```
dt = 100 s
dt = 50 s
dt = 25 s
```

Smaller timesteps increase runtime but generally improve integration accuracy.

Energy drift is one of the main diagnostics for deciding whether a timestep is sufficiently small.

Common Troubleshooting

CMake Cannot Find GLFW or GLAD

Make sure the standalone vcpkg installation contains the packages:

```
& "C:\Users\Maddie\Documents\GitHub\vcpkg\vcpkg.exe" list
```

Install them if necessary:

```
& "C:\Users\Maddie\Documents\GitHub\vcpkg\vcpkg.exe" install glfw3:x64-windows
glad:x64-windows
```

Make sure CMake uses:

```
C:/Users/Maddie/Documents/GitHub/vcpkg/scripts/buildsystems/vcpkg.cmake
```

and not the Visual Studio bundled vcpkg tree.

CMake Cached the Wrong vcpkg Toolchain

From the build directory:

```
Remove-Item CMakeCache.txt -ErrorAction SilentlyContinue; Remove-Item CMakeFiles -  
Recurse -Force -ErrorAction SilentlyContinue
```

Then configure again.

cl.exe Not Found

Launch a Visual Studio Build Tools developer environment with:

```
cmd /k ""C:\Program Files (x86)\Microsoft Visual  
Studio\2022\BuildTools\VC\Auxiliary\Build\vcvars64.bat""
```

nvcc Not Found

The current CUDA compiler is installed at:

```
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v13.3\bin\nvcc.exe
```

Verify with:

```
where.exe nvcc
```

GLAD Reports OpenGL Was Already Included

GLAD must be included before headers that include OpenGL.

The current `main.cu` starts with:

```
#define GLFW_INCLUDE_NONE  
  
#include <glad/glad.h>  
#include <GLFW/glfw3.h>  
  
#include <cuda_gl_interop.h>  
#include <cuda_runtime.h>
```

CUDA/OpenGL Registration Failure

The OpenGL context must exist before calling:

```
cudaGraphicsGLRegisterBuffer(...)
```

CUDA and OpenGL must also operate on compatible GPU devices.

Potential Extensions

Possible future improvements include:

- Barnes-Hut gravitational approximation
- Octree spatial subdivision
- CPU reference implementation
- CPU/GPU correctness comparison
- Double-precision body positions
- Adaptive timestep integration
- Collision detection
- Collision merging
- Multiple body types
- Initial orbital systems
- Galaxy initial conditions
- GPU-only history
- Reduced-memory rewind history
- Benchmark mode without visualization
- Automated block-size benchmark scripts
- CUDA streams
- Kernel fusion
- Structure-of-arrays memory layout
- Multi-GPU simulation
- Improved shaders
- Particle coloring based on mass or velocity
- Trails
- Camera translation
- Simulation-state save/load
- Output files for later analysis

Scaling Beyond Direct N-Body

The current direct algorithm is intentionally simple and exact with respect to pair evaluation, but its $O(N^2)$ scaling eventually becomes expensive.

A likely future direction is Barnes-Hut.

Barnes-Hut groups distant bodies together and approximates their gravitational influence, reducing the expected computational complexity toward:

```
 $O(N \log N)$ 
```

This would allow substantially larger simulations while retaining direct interactions for nearby bodies.

License

This repository does not currently include a formal license file.

If the project is distributed or published for reuse, an explicit license should be added.

Summary

This repository is a CUDA learning, physics, visualization, and optimization project built around gravitational N-body simulation.

It demonstrates:

- CUDA kernel development
- GPU memory management
- Shared-memory tiling
- Warp- and block-level performance tuning
- Numerical integration
- Physical conservation diagnostics
- CUDA/OpenGL interoperability
- Interactive visualization
- GPU benchmarking

The current direct-force implementation is intentionally straightforward enough to study while still providing several realistic GPU-performance and numerical-physics problems to optimize.